Assignment 3 – Diffusion Playground

Sampling, Editing, and Illusions with a Pretrained Text-to-Image Diffusion Model

Course: Neural Graphics (3683-8901)
Released: July 5, 2026
Due: July 26, 2026, 23:59

Introduction

In Assignment 2 you built and trained a diffusion model from scratch on MNIST. In this assignment you will work with a much larger, pretrained text-to-image diffusion model – DeepFloyd IF – and use it as a building block for a variety of sampling and editing tasks. Rather than training a model, your job here is to correctly implement the *sampling and editing mathematics* on top of an existing, frozen model: the forward and reverse diffusion process, classifier-free guidance, image editing via partial noising (SDEdit), inpainting, and two "diffusion illusions" – visual anagrams and hybrid images (the latter offered as an optional bonus) – that exploit the flexibility of the reverse process.

Every formula you need was covered in lecture and in Assignment 2; this assignment is about applying that same DDPM machinery to a real pretrained model and building intuition for what each piece of the sampling loop is actually doing.

Skeleton Files

Download `Assignment3.ipynb` (this is a single self-contained notebook). Cells marked `# ===== your code here! =====` (or `raise NotImplementedError`) are where you implement something.

Environment Setup

Google Colab

Open `Assignment3.ipynb` directly in Google Colab and enable a GPU runtime via **Runtime** → **Change runtime type** → **T4 GPU**.

HuggingFace Access

This assignment uses **DeepFloyd IF**, a pretrained text-to-image diffusion model hosted on HuggingFace. Before you can load it you must:

1. Create a free account at `https://huggingface.co` if you don't already have one.
2. Visit the DeepFloyd/IF-I-L-v1.0 model page and accept the license agreement (click through the gated-access prompt).
3. Generate a read-access token from your HuggingFace account settings (Settings → Access Tokens).
4. Run the login cell near the top of the notebook and paste your token into the widget that appears.

Do this well before the deadline – license approval is usually instant but occasionally takes a few hours, and you cannot proceed without it.

Text Prompt Embeddings

DeepFloyd IF conditions on embeddings from a large T5 text encoder. Running that encoder yourself is optional and not required for this assignment: the notebook downloads a precomputed dictionary of prompt embeddings (`prompt_embeds_dict.pth`) covering a fixed set of prompts, and this is the intended way to complete every deliverable below. Run the provided cell in Part 0 and print the available keys before deciding which prompts to use for each section – the exact prompt strings matter (they must match a dictionary key exactly).

Part 0: Loading the Model

Run the setup cells in order: installing dependencies, logging into HuggingFace, and loading the two DeepFloyd IF stages (`stage_1` and `stage_2`). Loading both stages may take a couple of minutes the first time, since the notebook needs to download the pretrained weights.

Take a moment to look at the sample outputs from the provided prompts and note the difference between the 64×64 output of stage 1 and the 256×256 upsampled output of stage 2.

Task 0.1. Generate samples using at least 3 different text prompts from the precomputed dictionary, and display both the stage-1 and stage-2 outputs for each. Also generate one image using a very low `num_inference_steps` (e.g. 5) and one using a much higher value (e.g. 50) with the same prompt, and compare the visual quality.

Part 1: Sampling Loop Mechanics

1.1. The Forward Process

Recall from lecture (and Assignment 2) that the forward diffusion process adds noise to a clean image x_0 according to

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

The notebook provides precomputed values of $\bar{\alpha}_t$ (`alphas_cumprod`) taken directly from the pretrained model's noise schedule, as well as a working implementation of `forward(im, t)` – this uses the same formula you already implemented in Assignment 2 (just against a schedule pulled from the pretrained model instead of one you build yourself).

Task 1.1. Read the provided `forward(im, t)` function and confirm you understand each line. Use it to display a test image noised at $t \in \{250, 500, 750\}$.

1.2. Denoising: One-Step vs. Iterative

The DeepFloyd UNet can estimate the noise $\hat{\epsilon}_\theta(x_t, t)$ present in a noisy image in a single forward pass, which lets us estimate the original clean image directly:

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \hat{\epsilon}_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

This one-step estimate performs poorly at high t, because the model has to guess the entire clean image in a single shot. Instead, we can iteratively remove a small amount of noise at a time,

following the DDPM reverse step:

$$x_{t'} = \frac{\sqrt{\bar{\alpha}_{t'}}\,\beta_t}{1-\bar{\alpha}_t}\,\hat{x}_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t'})}{1-\bar{\alpha}_t}\,x_t + \sigma_t z,$$

where $t' < t$ is the next (less noisy) timestep in a strided schedule.

Task 1.2a. Implement one-step denoising using the UNet's noise estimate at $t \in \{250, 500, 750\}$, and display the results.

Task 1.2b. Construct a strided list of timesteps (`strided_timesteps`) from ∼990 down to 0 in steps of 30. Implement `iterative_denoise`. Starting from a test image noised to $i\_start = 10$, run the iterative denoising loop.

Task 1.2c. Compute the one-step estimate of that same noisy image ($i\_start = 10$) and display it side by side with the iterative result. Discuss in your writeup why iterative denoising outperforms one-step denoising at this noise level, despite using the exact same trained model.

1.3. Sampling: With and Without Classifier-Free Guidance (CFG)

Setting $i\_start = 0$ and starting from pure Gaussian noise turns `iterative_denoise` into a sampler. However, plain conditional sampling this way often produces low-quality images. Classifier-free guidance (CFG) extrapolates away from the unconditional noise estimate to improve quality at the cost of diversity:

$$\hat{\epsilon}_{\text{cfg}} = \hat{\epsilon}_u + \gamma\,(\hat{\epsilon}_c - \hat{\epsilon}_u),$$

where $\hat{\epsilon}_c$ and $\hat{\epsilon}_u$ are the conditional and unconditional noise estimates and γ is the guidance scale.

Task 1.3. Generate 5 images from scratch using the prompt `"a high quality photo"` **without** CFG (reusing `iterative_denoise`). Then implement `iterative_denoise_cfg` and generate 5 more images with the same prompt **with** CFG at $\gamma = 7$. Display both sets and discuss the visual difference in your writeup.

Part 2: Image Editing

2.1. SDEdit on Web and Hand-Drawn Images

Partially noising a real image and then running it through `iterative_denoise_cfg` (starting partway through the schedule instead of from pure noise) produces an edited image that stays loosely faithful to the input while injecting new content – this is the SDEdit technique. **No new function is needed for this task** – you are reusing `iterative_denoise_cfg` from Task 1.3, just changing what image you feed it and where in the schedule you start.

Task 2.1. For each $i\_start \in \{1, 3, 5, 7, 10, 20\}$: noise the provided test image to the timestep corresponding to that i_start, then run `iterative_denoise_cfg` starting from that timestep (instead of from pure noise). Display the resulting edit for every i_start so you can see the progression. Then repeat the exact same procedure on **one image of your own** – either downloaded from the web or hand-drawn (using the provided drawing widget, or any simple image editor if you are not running Colab interactively; you only need to complete one of these two options, doing both is fine but not required). Discuss how the strength of the edit changes with i_start.

2.2. Inpainting

Using a binary mask, we can force the reverse process to leave everything outside the mask unchanged (re-noised to match the current timestep) while letting the model freely denoise inside the mask. **This task does require new code**: unlike Task 2.1, there is no existing function that does this, so you will implement `inpaint` yourself (it will look similar to `iterative_denoise_cfg`, with one key addition: resetting everything outside the mask back to the original image at every step).

Task 2.2. Implement `inpaint`, then use it to inpaint the top of the provided test image (a mask for this is provided). Then inpaint a region of **one image of your choosing**, providing your own mask (any array of 0s and 1s the same size as your image; 1 marks the region the model is allowed to fill in).

2.3. Text-Conditional Image-to-Image Translation

Task 2.3. Repeat the SDEdit procedure from Task 2.1, but this time use a text prompt (other than `"a high quality photo"`) to steer the edit. Try $i\_start \in \{1, 3, 5, 7, 10, 20\}$ on the provided test image, and discuss how the choice of prompt changes what the model introduces into the image compared to the unconditional edit in Task 2.1.

Part 3: Diffusion Illusions

3.1. Visual Anagrams

By averaging the noise estimates of an image and its vertical flip – each conditioned on a *different* prompt – we can create a single image that looks like one thing right-side up and something else upside down:

$$\hat{\epsilon} = \tfrac{1}{2}\big(\hat{\epsilon}_\theta(x_t, t, p_1) + \text{flip}(\hat{\epsilon}_\theta(\text{flip}(x_t), t, p_2))\big).$$

This is the Visual Anagrams technique.

Task 3.1. Implement `make_flip_illusion` and generate at least one visual anagram. The prompt pair `"an oil painting of an old man"` / `"an oil painting of people around a campfire"` is a reliable starting point, but you are encouraged to try other pairs from the precomputed dictionary.

3.2. Hybrid Images (Bonus)

This subsection is optional and worth bonus credit – there is no penalty for skipping it.

A *hybrid image* is a classic image-compositing trick (Oliva, Kelly & Torralba, 2006): take the low spatial frequencies of one image and the high spatial frequencies of another, add them together, and the result reads as image A from a distance (or with blurred vision) and image B up close – because human perception weights low frequencies more at a distance and high frequencies more up close.

We can create a diffusion-model version of this (following Factorized Diffusion) by applying the same low-pass/high-pass trick to the model's *noise estimates*, rather than to pixels directly. Given two prompts p_1 and p_2, we combine their noise estimates as

$$\hat{\epsilon} = f_{\text{low}}(\hat{\epsilon}_\theta(x_t, t, p_1)) + f_{\text{high}}(\hat{\epsilon}_\theta(x_t, t, p_2)),$$

where f_{low} is a low-pass filter (e.g. Gaussian blur) and f_{high} is a high-pass filter (the image minus its own low-pass version), and use $\hat{\epsilon}$ in place of a single noise estimate in your reverse-process update.

Bonus Task. Implement `make_hybrids` and generate at least one hybrid image. If you attempt this, discuss in your writeup which prompt dominates the image at a distance versus up close, and why.

Hint: hybrid images work best when the two prompts describe subjects with a similar coarse silhouette (e.g. a skull and a waterfall, both roughly rounded/vertical), since the low-frequency prompt defines the overall shape and the high-frequency prompt only needs to fill in texture on top of it.

Submission

Submit a single `.zip` file via the course portal containing:

- Your completed `Assignment3.ipynb` (with all outputs run and visible)
- Your PDF writeup

Your writeup should include, for every task above: the requested generated images, and written answers to every discussion question. Also include the specific text prompts you used for each generation task, since results are prompt-dependent.

Acknowledgment: this assignment adapts the DeepFloyd IF diffusion-sampling exercises from UC Berkeley's CS 180/280 course materials.